

Evaluation of Functions

CHAPTER 5

5.0 Introduction

The purpose of this chapter is to acquaint you with a selection of the techniques that are frequently used in evaluating functions. In Chapter 6, we will apply and illustrate these techniques by giving routines for a variety of specific functions. The purposes of this chapter and the next are thus mostly congruent. Occasionally, however, the method of choice for a particular special function in Chapter 6 is peculiar to that function. By comparing this chapter to the next one, you should get some idea of the balance between “general” and “special” methods that occurs in practice.

Insofar as that balance favors general methods, this chapter should give you ideas about how to write your own routine for the evaluation of a function that, while “special” to you, is not so special as to be included in Chapter 6 or the standard function libraries.

CITED REFERENCES AND FURTHER READING:

Fike, C.T. 1968, *Computer Evaluation of Mathematical Functions* (Englewood Cliffs, NJ: Prentice-Hall).

Lanczos, C. 1956, *Applied Analysis*; reprinted 1988 (New York: Dover), Chapter 7.

5.1 Polynomials and Rational Functions

A polynomial of degree N is represented numerically as a stored array of coefficients, $c[j]$ with $j = 0, \dots, N$. We will always take $c[0]$ to be the constant term in the polynomial and $c[N]$ the coefficient of x^N ; but of course other conventions are possible. There are two kinds of manipulations that you can do with a polynomial: *numerical* manipulations (such as evaluation), where you are given the numerical value of its argument, or *algebraic* manipulations, where you want to transform the coefficient array in some way without choosing any particular argument. Let's start with the numerical.

We assume that you know enough *never* to evaluate a polynomial this way:

```
p=c[0]+c[1]*x+c[2]*x*x+c[3]*x*x*x+c[4]*x*x*x*x;
```

or (even worse!),

```
p=c[0]+c[1]*x+c[2]*pow(x,2.0)+c[3]*pow(x,3.0)+c[4]*pow(x,4.0);
```

Come the (computer) revolution, all persons found guilty of such criminal behavior will be summarily executed, and their programs won't be! It is a matter of taste, however, whether to write

```
p=c[0]+x*(c[1]+x*(c[2]+x*(c[3]+x*c[4])));
```

or

```
p=((c[4]*x+c[3])*x+c[2])*x+c[1])*x+c[0];
```

If the number of coefficients $c[0..n-1]$ is large, one writes

```
p=c[n-1];
for(j=n-2;j>=0;j--) p=p*x+c[j];
```

or

```
p=c[j=n-1];
while (j>0) p=p*x+c[--j];
```

We can formalize this by defining a function object (or functor) that binds a reference to an array of coefficients and endows them with a polynomial evaluation function,

```
poly.h struct Poly {
Polynomial function object that binds a reference to a vector of coefficients.
    VecDoub &c;
    Poly(VecDoub &cc) : c(cc) {}
    Doub operator() (Doub x) {
        Returns the value of the polynomial at x.
        Int j;
        Doub p = c[j=c.size()-1];
        while (j>0) p = p*x + c[--j];
        return p;
    }
};
```

which allows you to write things like

```
y = Poly(c)(x);
```

where c is a coefficient vector.

Another useful trick is for evaluating a polynomial $P(x)$ and its derivative $dP(x)/dx$ simultaneously:

```
p=c[n-1];
dp=0.;
for(j=n-2;j>=0;j--) {dp=dp*x+p; p=p*x+c[j];}
```

or

```
p=c[j=n-1];
dp=0.;
while (j>0) {dp=dp*x+p; p=p*x+c[--j];}
```

which yields the polynomial as p and its derivative as dp using coefficients $c[0..n-1]$.

The above trick, which is basically *synthetic division* [1,2], generalizes to the evaluation of the polynomial and nd of its derivatives simultaneously:

```
void ddpoly(VecDoub_I &c, const Doub x, VecDoub_O &pd)
```

poly.h

Given the coefficients of a polynomial of degree nc as an array $c[0..nc]$ of size $nc+1$ (with $c[0]$ being the constant term), and given a value x , this routine fills an output array pd of size $nd+1$ with the value of the polynomial evaluated at x in $pd[0]$, and the first nd derivatives at x in $pd[1..nd]$.

```
{
    Int nnd,j,i,nc=c.size()-1,nd=pd.size()-1;
    Doub cnst=1.0;
    pd[0]=c[nc];
    for (j=1;j<nd+1;j++) pd[j]=0.0;
    for (i=nc-1;i>=0;i--) {
        nnd=(nd < (nc-i) ? nd : nc-i);
        for (j=nnd;j>0;j--) pd[j]=pd[j]*x+pd[j-1];
        pd[0]=pd[0]*x+c[i];
    }
    for (i=2;i<nd+1;i++) {          After the first derivative, factorial constants come in.
        cnst *= i;
        pd[i] *= cnst;
    }
}
```

As a curiosity, you might be interested to know that polynomials of degree $n > 3$ can be evaluated in *fewer* than n multiplications, at least if you are willing to precompute some auxiliary coefficients and, in some cases, do some extra addition. For example, the polynomial

$$P(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 \quad (5.1.1)$$

where $a_4 > 0$, can be evaluated with three multiplications and five additions as follows:

$$P(x) = [(Ax + B)^2 + Ax + C][(Ax + B)^2 + D] + E \quad (5.1.2)$$

where A, B, C, D , and E are to be precomputed by

$$\begin{aligned} A &= (a_4)^{1/4} \\ B &= \frac{a_3 - A^3}{4A^3} \\ D &= 3B^2 + 8B^3 + \frac{a_1A - 2a_2B}{A^2} \\ C &= \frac{a_2}{A^2} - 2B - 6B^2 - D \\ E &= a_0 - B^4 - B^2(C + D) - CD \end{aligned} \quad (5.1.3)$$

Fifth-degree polynomials can be evaluated in four multiplies and five adds; sixth-degree polynomials can be evaluated in four multiplies and seven adds; if any of this strikes you as interesting, consult references [3-5]. The subject has something of the same flavor as that of fast matrix multiplication, discussed in §2.11.

Turn now to algebraic manipulations. You multiply a polynomial of degree $n - 1$ (array of range $[0..n-1]$) by a monomial factor $x - a$ by a bit of code like the following,

```
c[n]=c[n-1];
for (j=n-1;j>=1;j--) c[j]=c[j-1]-c[j]*a;
c[0] *= (-a);
```

Likewise, you divide a polynomial of degree n by a monomial factor $x - a$ (synthetic division again) using

```
rem=c[n];
c[n]=0.;
for(i=n-1;i>=0;i--) {
    swap=c[i];
    c[i]=rem;
    rem=swap+rem*a;
}
```

which leaves you with a new polynomial array and a numerical remainder `rem`.

Multiplication of two general polynomials involves straightforward summing of the products, each involving one coefficient from each polynomial. Division of two general polynomials, while it can be done awkwardly in the fashion taught using pencil and paper, is susceptible to a good deal of streamlining. Witness the following routine based on the algorithm in [3].

poly.h

```
void poldiv(VecDoub_I &u, VecDoub_I &v, VecDoub_O &q, VecDoub_O &r)
Divide a polynomial u by a polynomial v, and return the quotient and remainder polynomials
in q and r, respectively. The four polynomials are represented as vectors of coefficients, each
starting with the constant term. There is no restriction on the relative lengths of u and v, and
either may have trailing zeros (represent a lower degree polynomial than its length allows). q
and r are returned with the size of u, but will usually have trailing zeros.
{
    Int k,j,n=u.size()-1,nv=v.size()-1;
    while (nv >= 0 && v[nv] == 0.) nv--;
    if (nv < 0) throw("poldiv divide by zero polynomial");
    r = u;
    q.assign(u.size(),0.);
    for (k=n-nv;k>=0;k--) {
        q[k]=r[nv+k]/v[nv];
        for (j=nv+k-1;j>=k;j--) r[j] -= q[k]*v[j-k];
    }
    for (j=nv;j<=n;j++) r[j]=0.0;
}
```

5.1.1 Rational Functions

You evaluate a rational function like

$$R(x) = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1x + \cdots + p_\mu x^\mu}{q_0 + q_1x + \cdots + q_\nu x^\nu} \quad (5.1.4)$$

in the obvious way, namely as two separate polynomials followed by a divide. As a matter of convention one usually chooses $q_0 = 1$, obtained by dividing the numerator and denominator by any other q_0 . In that case, it is often convenient to have both sets of coefficients, omitting q_0 , stored in a single array, in the order

$$(p_0, p_1, \dots, p_\mu, q_1, \dots, q_\nu) \quad (5.1.5)$$

The following object encapsulates a rational function. It provides constructors from either separate numerator and denominator polynomials, or a single array like (5.1.5) with explicit values for $n = \mu + 1$ and $d = \nu + 1$. The evaluation function makes `Ratfn` a functor, like `Poly`. We'll make use of this object in §5.12 and §5.13.

```

struct Ratfn {
Function object for a rational function.
    VecDoub cofs;
    Int nn,dd;
                                Number of numerator, denominator coefficients.

    Ratfn(VecDoub_I &num, VecDoub_I &den) : cofs(num.size()+den.size()-1),
    nn(num.size()), dd(den.size()) {}
    Constructor from numerator, denominator polynomials (as coefficient vectors).
        Int j;
        for (j=0;j<nn;j++) cofs[j] = num[j]/den[0];
        for (j=1;j<dd;j++) cofs[j+nn-1] = den[j]/den[0];
    }

    Ratfn(VecDoub_I &cofs, const Int n, const Int d) : cofs(cofs), nn(n),
    dd(d) {}
    Constructor from coefficients already normalized and in a single array.

    Doub operator() (Doub x) const {
    Evaluate the rational function at x and return result.
        Int j;
        Doub sumn = 0., sumd = 0.;
        for (j=nn-1;j>=0;j--) sumn = sumn*x + cofs[j];
        for (j=nn+dd-2;j>=nn;j--) sumd = sumd*x + cofs[j];
        return sumn/(1.0+x*sumd);
    }
};

```

poly.h

5.1.2 Parallel Evaluation of a Polynomial

A polynomial of degree N can be evaluated in about $\log_2 N$ parallel steps [6]. This is best illustrated by an example, for example with $N = 5$. Start with the vector of coefficients, imagining appended zeros:

$$c_0, \quad c_1, \quad c_2, \quad c_3, \quad c_4, \quad c_5, \quad 0, \quad \dots \quad (5.1.6)$$

Now add the elements by pairs, multiplying the second of each pair by x :

$$c_0 + c_1x, \quad c_2 + c_3x, \quad c_4 + c_5x, \quad 0, \quad \dots \quad (5.1.7)$$

Now the same operation, but with the multiplier x^2 :

$$(c_0 + c_1x) + (c_2 + c_3x)x^2, \quad (c_4 + c_5x) + (0)x^2, \quad 0 \quad \dots \quad (5.1.8)$$

And a final time with multiplier x^4 :

$$[(c_0 + c_1x) + (c_2 + c_3x)x^2] + [(c_4 + c_5x) + (0)x^2]x^4, \quad 0 \quad \dots \quad (5.1.9)$$

We are left with a vector of (active) length 1, whose value is the desired polynomial evaluation. You can see that the zeros are just a bookkeeping device for taking care of the case where the active subvector has an odd length; in an actual implementation you can avoid most operations on the zeros. This parallel method generally has better roundoff properties than the standard sequential coding.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 183, 190.[1]